

Day 1 Foundations & First Deployment

Participant Manual

Platform	AxisPay (fictional)
Kubernetes	v1.36
Environment	Ubuntu 26.04 LTS · Minikube
Built	14 August 2026

Every merchant, customer, card token, acquirer and transaction in this manual is fictional. No real institution is represented and no card number exists anywhere in this platform — only tokens. The platform is **not** PCI-DSS compliant and is not presented as such; it uses PCI-shaped constraints to make security controls consequential in a training context.

Day 3 — State, Configuration and Data

AxisPay · Kubernetes Comprehensive · Participant Manual, Chapter 3

What changed today

Yesterday	Today
Config hard-coded in eleven manifests	Two ConfigMaps
JWT signing key in a plain env var	A Secret, mounted read-only
Every payment lost on pod restart	PostgreSQL with 5,000 transactions
Fraud counters per-replica	Shared Redis counters
No storage at all	PV, PVC, StorageClass, three StatefulSets
Containers run as root	UID 10001, read-only rootfs, zero capabilities

3.1 ConfigMaps

1. What it is

A named collection of key/value pairs, stored in the cluster, that pods can read as environment variables or as files.

2. Why it exists

Configuration changes far more often than code. Baking it into a workload definition means a code review and a pipeline run to change a log level.

3. The business problem

AxisPay's fraud team wants the review threshold lowered from 45 to 40 for the weekend. Today that edits `fraud-service`'s Deployment — a change to the payment path — for a number. Meanwhile `LOG_LEVEL` is duplicated across eleven manifests, so turning on debug logging during an incident means eleven edits at 03:00.

4. How it works

Two consumption modes, and they behave completely differently:

Mode	Behaviour	Consequence
<code>env</code> / <code>envFrom</code>	Values copied into the process environment once , at container creation	A snapshot . Changing the ConfigMap does nothing to a running pod — no error, no event.
<code>volumeMounts</code>	Each key projected as a file, updated in place by the kubelet	Changes appear within ~60 s. But Kubernetes updated the <i>file</i> ; the application must re-read it.

The `subPath` exception. Mounting a single key with `subPath` produces a real file rather than a symlink into the projected directory — and it **never updates**. This is documented, and it is the most common "why is my mounted config not changing" question.

5. Internal architecture

A ConfigMap is a plain object in etcd. For volume mounts the kubelet writes the data into a `tmpfs` directory on the node and symlinks the keys into it. Updates are atomic: the kubelet writes a new timestamped directory and swings the symlink, so a reader never sees a half-written file.

Size limit: **1 MiB**, inherited from etcd's practical object limit. AxisPay's 10 KiB schema fits; the 8 MB seed file does not, which is why it is piped into `psql` instead.

6. Component interactions

```
you          kubectl apply configmap
API server   stores in etcd
kubelet      (env)    reads once at container create — snapshot
kubelet      (volume) watches; on change writes a new dir and swings the symlink
your app     (volume) must re-read the file to notice
```

7. Enterprise example

A payments platform keeps one ConfigMap per environment holding endpoints, timeouts and feature flags, and a separate one for risk thresholds mounted as a volume. Application config is a snapshot (deliberate, releasable); risk tuning is live (deliberate, operational). The split is a design decision recorded in the runbook, not an accident.

8. Real-world analogy

An env var is a **printed** copy of the price list handed to a cashier at the start of a shift. A mounted volume is a **screen** on the wall showing the current list — but the cashier still has to look up.

9. Best practices

Practice	Reason
Use env for values read once at startup	A snapshot is honest about how the value is used
Use a volume for values operations must change live	No rollout on the payment path
Add a checksum annotation to the pod template	Config changes become releases: visible in <code>rollout history</code> , rollback-able
Never put credentials in a ConfigMap	It has none of a Secret's (already limited) protections
Keep ConfigMaps small	1 MiB limit; large ones bloat every watch
Consider <code>immutable: true</code> at scale	Removes the kubelet's watch and reduces API server load

10. Common mistakes

Mistake	Symptom
Expecting env vars to update live	Nothing changes; nothing warns you
Using <code>subPath</code> and expecting updates	The file never changes
Key renamed	Env var silently absent, app falls back to a default — this is INC-3b
Credentials in a ConfigMap	Readable by anyone with <code>get configmaps</code>
ConfigMap over 1 MiB	Rejected at creation

11. Security considerations

Threat	Control	Residual risk
Credentials placed in a ConfigMap	Policy + review; use Secrets	Secrets are barely better without RBAC
ConfigMap readable cluster-wide	RBAC per namespace (Day 5)	Namespace admins can read all of them
Config injection changing behaviour	Restrict <code>update</code> on ConfigMaps in production	A compromised CI account can still change them

12. Performance considerations

Each mounted ConfigMap adds a watch per pod. At thousands of pods this becomes measurable API server load — which is what `immutable: true` addresses. Volume updates are not instant: they follow the kubelet sync period, typically up to 60 seconds.

13. High availability

ConfigMaps are metadata and inherit control-plane HA. Note the failure mode: if a ConfigMap is deleted, **running pods keep working** (env vars are already copied, volumes already projected), but **new pods fail to start** with `CreateContainerConfigError`. The damage appears at the next scale-up or rollout, not immediately.

14. Disaster recovery

Trivially reproducible from manifests — provided they are in Git. A ConfigMap edited live with `kubectl edit` and never committed is exactly the drift that makes a rebuild fail.

15. Monitoring

Signal	Why
<code>CreateContainerConfigError</code> events	Missing ConfigMap or key
Pods not restarted after a config change	The checksum annotation is missing
ConfigMap size approaching 1 MiB	Will be rejected

16. Troubleshooting

Symptom	Cause	Command	Fix
<code>CreateContainerConfigError</code>	ConfigMap or key missing	<code>kubectl describe pod → Events</code>	Create it; check the key name
Env var not updating	Snapshot	—	Restart, or use a volume
Mounted file not updating	<code>subPath</code> , or <code>immutable</code>	<code>kubectl get cm -o yaml</code>	Remove <code>subPath</code>
App reads a default instead of your value	Key name mismatch	<code>kubectl exec -- printenv \ sort</code>	Compare with the ConfigMap

Interview questions

- You change a ConfigMap. Do running pods see it?** *Depends how it is consumed. As env vars, no — they are a snapshot from container creation. As a mounted volume, yes, within about 60 seconds — but the application must re-read the file.*
- How do you make a config change trigger a rollout?** *Put a hash of the ConfigMap in a pod-template annotation. Any change alters the template, creating a new ReplicaSet and rolling the pods. Helm does this automatically.*
- Why is there a 1 MiB limit?** *(senior) It comes from etcd's practical object size. It matters in practice: schema files fit, seed data does not, and large ConfigMaps are replicated to every watching kubelet.*
- When would you set `immutable: true`?** *(senior) At scale. It tells the kubelet not to watch the object, cutting API server load significantly across thousands of pods. The cost is that a change requires creating a new ConfigMap and updating the reference — which some teams consider a feature, because it makes config changes explicit releases.*

3.2 Secrets

1. What it is

An object for holding small pieces of sensitive data — passwords, tokens, keys — that pods consume as environment variables or files.

2. Why it exists

To keep credentials out of images and manifests, and to give the cluster a distinct object type that RBAC and encryption-at-rest can be applied to.

3. The business problem

Since Day 1 the JWT signing key has been a plain environment variable in `auth-service`'s Deployment. Anyone with `get pods` in `axispay-edge` can read it with `kubectl describe`. With that key they can mint a valid token for **any** merchant and authorise payments as them.

4. How it works

A Secret is base64-encoded. It is not encrypted.

```
kubectl get secret axispay-db-credentials -n axispay-data \
  -o jsonpath='{.data.POSTGRES_PASSWORD}' | base64 -d
```

There is no key, no algorithm, nothing to break. `base64 -d` is a formatting change. Anyone who can read the object can read every value in it.

What actually protects a Secret:

Control	What it does
RBAC	Restricts who may <code>get</code> Secrets, per namespace. <i>The real boundary.</i>
Encryption at rest	etcd encrypts Secret values on disk. Without it, an etcd backup is a plaintext credential dump.
Mount as a volume, not env	Fewer accidental leak paths (see §9)
External secret manager	Short-lived, rotatable, audited credentials. The production answer.

5. Internal architecture

Secret volumes are mounted as **tmpfs** — RAM-backed, never written to the node's disk. Files default to mode `0644`; AxisPay sets `0400`. Like ConfigMaps, updates are projected atomically.

`stringData` accepts plaintext in a manifest and Kubernetes base64-encodes it on write. It is more readable than pre-encoded `data:` and exactly as (in)secure.

6. Component interactions

Identical to ConfigMaps, with one difference: the kubelet only fetches Secrets a pod actually references, and holds them in memory rather than on disk.

7. Enterprise example

A bank runs External Secrets Operator against HashiCorp Vault. Kubernetes Secrets exist, but are **projections** with a 1-hour TTL, refreshed automatically. Nothing sensitive is ever committed to Git; the manifest contains only a reference. Rotation is a Vault operation and the cluster follows within the hour.

8. Real-world analogy

A Secret is a **sealed envelope in an unlocked filing cabinet**. The envelope stops casual glancing; the lock on the cabinet (RBAC) is what actually protects it. Calling the envelope "encryption" is the mistake.

9. Best practices

Practice	Reason
Prefer volumes over env vars	Env vars are inherited by children, appear in <code>/proc/*/environ</code> , and are captured by crash handlers
Set <code>defaultMode: 0400</code>	Read-only, owner only
Never commit a Secret manifest to Git	Use sealed-secrets or an external manager
Enable etcd encryption at rest	Otherwise a backup is a credential dump
One Secret per concern	Limits blast radius and makes RBAC meaningful
Support two valid keys during rotation	Otherwise rotation is an outage
Restrict <code>get secrets</code> tightly	This is the actual control (Day 5)

10. Common mistakes

Mistake	Symptom
Believing Secrets are encrypted	Credentials exposed in an etcd backup
Using a plain <code>value:</code> instead of <code>secretKeyRef</code>	Visible in <code>kubectl describe pod</code>
Committing Secret manifests	Credentials in Git history forever
Trailing newline from <code>--from-file</code>	Auth fails with a correct-looking password
Rotating without a restart	Env vars are snapshots
One giant Secret for everything	Any pod needing one value gets all of them

11. Security considerations

Threat	Control	Residual risk
<code>kubectl get secret</code> by an over-privileged user	RBAC	Cluster-admin can always read them
etcd backup stolen	Encryption at rest + KMS	Key management becomes the problem
Env var captured by an error tracker	Mount as a volume	Deliberate exfiltration still possible
Secret in Git history	Sealed secrets / external manager	History rewriting is painful
Node compromise	tmpfs limits disk exposure	An attacker with root on the node can read process memory

The honest summary: today's work is a real improvement — reading the JWT key now needs `get secrets` as well as `get pods`. It is not encryption, and the boundary is RBAC.

12. Performance considerations

Negligible. Secrets are small by design (1 MiB limit). The only consideration at scale is the same watch load as ConfigMaps.

13. High availability

Same as ConfigMaps: deleting a Secret does not break running pods, but new pods fail with `CreateContainerConfigError`. The failure surfaces at the next rollout.

14. Disaster recovery

Secrets are the one thing you cannot restore from Git, because they should not be in Git. Recovery means an external secret manager, a sealed-secrets controller with its private

key backed up separately, or a documented manual re-creation procedure. Deciding which — before an incident — is the actual DR work.

15. Monitoring

Signal	Why
Audit log: <code>get secrets</code> by unexpected principals	Credential access is worth alerting on
<code>CreateContainerConfigError</code>	Missing Secret
Secret age	Rotation overdue

16. Troubleshooting

Symptom	Cause	Command	Fix
<code>CreateContainerConfigError</code>	Secret or key missing	<code>kubectl describe pod</code>	Create it; check the key
Auth fails with a correct password	Trailing newline	<code>... \\\ base64 -d \\\ xxd \\\ tail -1</code>	Use <code>--from-literal</code> or <code>printf</code>
Permission denied reading a mounted secret	<code>defaultMode</code> vs <code>runAsUser</code>	<code>kubectl exec --ls -la</code>	Align <code>defaultMode</code> and <code>fsGroup</code>
Rotation had no effect	Env var snapshot	—	<code>kubectl rollout restart</code>
Value visible in <code>describe</code>	Plain <code>value:</code> used	<code>kubectl get deploy -o yaml</code>	Use <code>secretKeyRef</code>

Interview questions

- 1. Is a Kubernetes Secret encrypted?** *No. base64-encoded, which is an encoding, not encryption. What protects it is RBAC on who may read the object, plus etcd encryption at rest so a backup is not a plaintext credential dump.*
- 2. Why prefer a volume over an environment variable?** *Env vars are inherited by every child process, appear in `/proc/<pid>/environ`, and are routinely captured whole by crash handlers and error-tracking SDKs. A file is read once by the code that needs it, and Secret volumes are tmpfs so they never touch the node's disk.*
- 3. How would you rotate a signing key with no downtime?** *(senior) The application must accept two valid keys during a rotation window: add the new key, deploy so both are trusted, switch signing to the new one, wait for old tokens to expire, then remove the old key. Kubernetes can deliver the material but cannot make the rotation safe — that is an application design decision.*
- 4. How do you keep Secrets out of Git while still doing GitOps?** *(senior) Sealed Secrets (encrypt with a controller's public key; only the in-cluster controller can decrypt) or External Secrets Operator (Git holds a reference; the operator fetches from Vault/*

AWS/GCP and projects a short-lived Secret). The second is generally preferred because rotation is handled outside the cluster.

3.3 The storage model

1. What it is

A set of objects that let a pod use storage which outlives it: **Volume** (ephemeral), **PersistentVolume** (a real piece of storage), **PersistentVolumeClaim** (a request for one), **StorageClass** (a recipe for making them on demand).

2. Why it exists

A container's filesystem is part of the container. Delete the pod and it is gone. Anything with a memory needs storage decoupled from the pod lifecycle.

3. The business problem

AxisPay must retain transaction records for seven years and reconcile them nightly. You cannot reconcile against memory, and a Python dictionary is not a system of record.

4. How it works

The **claim/bind contract** separates supply from demand:

```
Storage admin (or a provisioner) -> PersistentVolume  "here is 2Gi, RWO, Retain"
Developer                        -> PersistentVolumeClaim "I need 1Gi, RWO"
Control plane                    -> BIND if capacity, access mode and class all agree
Pod                              -> mounts the CLAIM, never the volume
```

That indirection is why the same manifest runs on Minikube hostPath, AWS EBS and a corporate SAN unchanged.

Access modes — the most misread field in Kubernetes:

Mode	Means	Does mean
<code>ReadWriteOnce</code>	One NODE may mount it read-write	~~one pod~~ — pods on the <i>same node</i> can share it
<code>ReadOnlyMany</code>	Many nodes, read-only	
<code>ReadWriteMany</code>	Many nodes, read-write. Needs NFS/CephFS.	Most block storage physically cannot do this
<code>ReadWriteOncePod</code>	Genuinely one pod	The mode people usually meant by RWO

`ReadWriteOncePod` exists *because* the misreading above was so common.

Reclaim policy — what happens to the DATA when the claim is deleted:

Policy	Effect	Right for
<code>Delete</code>	Volume and data destroyed	Caches, scratch, CI
<code>Retain</code>	PV goes <code>Released</code> , data kept, manual reclaim	Anything you cannot recreate

5. Internal architecture

CSI splits into two halves:

- **Controller plugin** — one per cluster. Talks to the storage API: "create a 5Gi volume". A cloud API call.
- **Node plugin** — a DaemonSet on every node. Attaches, formats and mounts. A kernel operation on one machine.

They are separate because creating a volume and mounting it are completely different operations in different places.

`volumeBindingMode` decides *when* binding happens:

Mode	Sequence	Result
<code>Immediate</code>	Volume created on some node → scheduler later picks a node → they may disagree	<code>volume node affinity conflict</code> — pod unschedulable forever
<code>WaitForFirstConsumer</code>	PVC stays Pending → scheduler picks a node → volume created there	Storage and scheduling agree by construction

On a multi-node cluster with node-local storage this is not an optimisation; it is the difference between working and not.

6. Component interactions

```
PVC created          -> provisioner watches
(WaitForFirstConsumer) -> nothing yet; PVC stays Pending
pod created          -> scheduler filters and scores, picks a node
provisioner          -> creates the volume ON THAT NODE, creates the PV, binds
CSI node plugin      -> attaches and mounts into the pod's namespace
kubelet              -> applies fsGroup (chowns) so a non-root process can write
```

7. Enterprise example

A bank uses a CSI driver against its SAN, with three StorageClasses: `gold` (replicated NVMe, `Retain`), `silver` (SSD, `Retain`), `bronze` (spinning, `Delete`). Developers request a class by name; the storage team owns what each means. Nobody writes a PersistentVolume by hand.

8. Real-world analogy

A PVC is a **hotel booking**: "one double room, three nights". The PV is the actual room. You never book room 412 — you book a *kind* of room and the system assigns one. A

StorageClass is the rate plan that decides which rooms you can be assigned and what happens at checkout.

9. Best practices

Practice	Reason
Retain for anything you cannot recreate	A deleted PVC must not destroy a ledger
<code>WaitForFirstConsumer</code> on node-local storage	Prevents the affinity conflict
Always set <code>fsGroup</code> for non-root workloads	Otherwise permission denied on a fresh volume
Never write PVs by hand in production	Use dynamic provisioning
Request the size you need, not "a bit extra"	Expansion is easy; shrinking is impossible
<code>allowVolumeExpansion: true</code>	Growing later without recreating
Give hostPath PVs <code>nodeAffinity</code>	Or the pod may schedule where the data is not

10. Common mistakes

Mistake	Symptom
Reading RWO as "one pod"	Designing around a guarantee you do not have
Requesting RWX on block storage	PVC Pending, terse event
Delete on a data volume	PVC deleted, data gone
No <code>fsGroup</code>	<code>permission denied</code> on a directory that looks fine
Immediate binding, multi-node	<code>volume node affinity conflict</code>
PGDATA at the mount root	<code>initdb: directory not empty (lost+found)</code>
Expecting a PVC to shrink	Not supported

11. Security considerations

Threat	Control	Residual risk
hostPath exposes the node filesystem	Forbid hostPath via Pod Security Admission	System workloads legitimately need it
Data at rest unencrypted	Encrypted StorageClass parameters	Key management
Deleted PVC destroys evidence	<code>Retain</code> + RBAC on <code>delete pvc</code>	Cluster-admin can still delete
Volume reused with old data	<code>Retain</code> requires explicit reclaim	Manual step can be done carelessly

12. Performance considerations

- **Node-local (hostPath) is fastest and least available.** Networked storage is the reverse.
- IOPS and throughput usually come from StorageClass **parameters**, not from Kubernetes.
- `WaitForFirstConsumer` adds a small delay to first start — worth it.
- Many small volumes cost more control-plane work than a few large ones.

13. High availability

Node-local storage pins a pod to a node. If that node is unavailable, the pod cannot run anywhere — and Kubernetes correctly refuses to start it elsewhere, because starting a database on an empty volume looks exactly like data loss.

Real answers, in order: networked storage that any node can mount · database-level replication so another replica holds the data · accepting the pinning and planning maintenance around it.

14. Disaster recovery

Back up	The volume contents — Kubernetes does not back up your data
How	Volume snapshots (<code>VolumeSnapshot</code>), or application-level dumps (<code>pg_dump</code>) shipped off-cluster
RPO	Your snapshot interval
RTO	Restore + replay time; test it
The trap	A PV is not a backup. <code>Retain</code> protects against an accidental delete, not against corruption, and not against a failed disk.

15. Monitoring

Metric	Threshold
<code>kube_persistentvolumeclaim_status_phase{phase="Pending"}</code>	> 2 min — this is INC-3a
<code>kubelet_volume_stats_available_bytes / capacity_bytes</code>	< 15%
<code>kube_persistentvolume_status_phase{phase="Failed"}</code>	Any
Released PVs accumulating	<code>Retain</code> needs manual reclaim

16. Troubleshooting

Symptom	Cause	Command	Fix
PVC Pending, "waiting for first consumer"	Normal	<code>kubectl describe pvc</code>	Create a pod
PVC Pending, "storageclass not found"	Typo or missing class	<code>kubectl get sc</code>	Fix <code>storageClassName</code> — INC-3a
PVC Pending, no matching PV	Size/mode/class mismatch	<code>kubectl describe pvc</code>	Align them
volume node affinity conflict	Volume on another node	<code>kubectl describe pod</code>	<code>WaitForFirstConsumer</code> , or networked storage
permission denied on the volume	No <code>fsGroup</code>	<code>kubectl exec -- ls -ld</code>	Set <code>fsGroup</code>
PV Released, will not rebind	<code>Retain</code> keeps <code>claimRef</code>	<code>kubectl get pv -o yaml</code>	Clear <code>claimRef</code> or recreate

Interview questions

- What does `ReadWriteOnce` actually mean?** *One node may mount it read-write. Several pods on that same node can share it. `ReadWriteOncePod` is the mode that genuinely means one pod, and it exists because RWO was so widely misread.*
- Why would you delay binding a volume?** `WaitForFirstConsumer` lets the scheduler choose the node first, then provisions the volume there. With `Immediate` the volume can be created on a node the scheduler will not use, and the pod is unschedulable forever with `volume node affinity conflict`.
- `Retain` versus `Delete`?** `Delete` destroys the volume and its data when the claim goes; right for caches and CI. `Retain` keeps it in a `Released` state for manual recovery; right for anything you cannot recreate. It is a decision made once in the `StorageClass`, and it is the difference between a recoverable mistake and a destroyed ledger.
- Your PVC has been Pending for five minutes. Broken or working as intended?** *(senior) Read the event. "waiting for first consumer" is correct behaviour — create a pod. "storageclass not found" or "no persistent volumes available" is genuinely broken. The status is identical; the event is everything.*
- Is a `PersistentVolume` a backup?** *(senior) No. It protects against pod loss, not against corruption, deletion or disk failure. Backups need snapshots or application-level dumps shipped off-cluster — and a tested restore. `Retain` is a safety catch, not a backup strategy.*

3.4 StatefulSets

1. What it is

A controller for workloads where each replica has an identity: a stable name, its own storage, and a defined start/stop order.

2. Why it exists

Deployments treat pods as interchangeable. Databases, queues and consensus systems need the opposite of that in all three dimensions.

3. The business problem

AxisPay needs PostgreSQL. A Deployment gives random names, shared-or-no storage, and replacement in any order — which produces two unrelated empty databases behind a load balancer.

4. How it works

Three guarantees a Deployment cannot provide:

Guarantee	Mechanism	Why it matters
Stable name	Ordinal naming: <code>postgres-0</code> , <code>postgres-1</code>	A replica can be told "your primary is <code>postgres-0</code> " and that stays true
Stable storage	<code>volumeClaimTemplates</code> → one PVC per replica, re-attached by ordinal	<code>postgres-0</code> always gets <code>data-postgres-0</code>
Stable order	<code>podManagementPolicy: OrderedReady</code> — start 0,1,2; terminate 2,1,0	A replica never starts before its primary

The **headless Service** (`clusterIP: None`) makes DNS return pod IPs rather than one virtual IP, giving `postgres-0.postgres.axispay-data.svc.cluster.local`. You cannot address one specific replica through a normal Service.

5. Internal architecture

`volumeClaimTemplates` creates PVCs named `<template>-<statefulset>-<ordinal>`. **These are not deleted when the StatefulSet is deleted** — deliberately, because deleting a workload must never silently destroy its data.

`podManagementPolicy` options: `OrderedReady` (default, sequential) or `Parallel` (all at once — right for workloads with no ordering requirement, such as a sharded cache).

6. Component interactions

```
StatefulSet controller -> creates PVC data-postgres-0
                        -> creates pod postgres-0, waits for Ready
                        -> creates PVC data-postgres-1, pod postgres-1 ...
headless Service       -> DNS: postgres-0.postgres.<ns>.svc.cluster.local
on reschedule          -> SAME name, SAME PVC, new node (if storage allows)
```

7. Enterprise example

Nobody runs a production PostgreSQL cluster on a bare StatefulSet. They use an **Operator** (CloudNativePG, Zalando, Crunchy) which builds on StatefulSets and adds what Kubernetes does not know: replication, leader election, failover, WAL archiving, backups. The StatefulSet provides identity and storage; the Operator provides the database.

8. Real-world analogy

A Deployment is a **pool of temps** — any of them will do. A StatefulSet is **named staff with assigned desks**: Desk 0 is always the same person's desk, their belongings are still in it when they return, and the induction order is fixed.

9. Best practices

Practice	Reason
Always pair with a headless Service	<code>serviceName</code> must match, or pods never get stable DNS
Never <code>Delete</code> reclaim policy	The PVCs are the data
Generous startup probes	<code>initdb</code> , recovery and schema load take time
Long <code>terminationGracePeriodSeconds</code>	Let a database checkpoint and close cleanly
Set <code>fsGroup</code>	Non-root databases cannot write to a fresh volume otherwise
Use an Operator for real clusters	Kubernetes knows nothing about replication
Readiness must run a real query	<code>pg_isready</code> succeeds while the server is still in recovery

10. Common mistakes

Mistake	Symptom
Database as a Deployment	Random names, lost data, no ordering
<code>serviceName</code> does not match	Pods never get stable DNS
Scaling to 3 and expecting a cluster	Three independent empty databases
Deleting the StatefulSet expecting cleanup	PVCs remain (deliberate)
<code>PGDATA</code> at the mount root	<code>initdb: directory not empty</code>
Missing <code>fsGroup</code>	Permission denied on a fresh volume
Aggressive liveness on a database	Restart during recovery, restart loop

11. Security considerations

Threat	Control	Residual risk
Database reachable from the whole cluster	NetworkPolicy (Day 4)	Needs a policy-enforcing CNI
Database running as root	<code>runAsNonRoot</code> + correct UID (postgres uses 999)	Image must support it
Credentials in the pod spec	Secret + <code>envFrom</code>	Secrets are not encrypted
Volume readable on the node	Encrypted storage class	Node compromise

12. Performance considerations

- **Ordered startup is serial.** Ten replicas mean ten sequential starts. `Parallel` is available when ordering does not matter.
- Storage class choice dominates database performance far more than anything Kubernetes does.
- Rolling updates go in reverse ordinal order, one at a time — safe, and slow.

13. High availability

A StatefulSet is not high availability. It gives identity and storage; it does not replicate data. Three PostgreSQL replicas in a StatefulSet are three separate databases unless something above them implements replication.

Real HA needs: an Operator managing replication and failover · anti-affinity so replicas are on different nodes (Day 4) · a PodDisruptionBudget (Day 4) · storage that survives node loss.

14. Disaster recovery

The PVCs are the data. Recovery means restoring volume snapshots or database dumps — Kubernetes objects alone restore an **empty** database with the right name.

Test the restore. A `pg_dump` that has never been restored is not a backup.

15. Monitoring

Metric	Threshold
<code>kube_statefulset_status_replicas_ready</code> vs <code>_replicas</code>	Gap > 5 min
<code>kubelet_volume_stats_available_bytes</code>	< 15%
Pod restart count on a data workload	Any — investigate
<code>kube_statefulset_status_observed_generation</code> lag	Controller not reconciling

16. Troubleshooting

Symptom	Cause	Command	Fix
<code>postgres-1</code> never starts	<code>OrderedReady</code> , pod 0 not Ready	<code>kubectl get pods</code>	Fix pod 0 first
Pod Pending after a node drain	Node-local volume elsewhere	<code>kubectl describe pod</code>	Uncordon, or networked storage
PVCs remain after deletion	Deliberate	<code>kubectl get pvc</code>	Delete explicitly if you mean it
<code>initdb: directory not empty</code>	<code>PGDATA</code> at the mount root	<code>kubectl logs</code>	Use a subdirectory
Permission denied on the data dir	No <code>fsGroup</code>	<code>kubectl exec -- ls -ld</code>	Set <code>fsGroup: 999</code>
Restart loop during recovery	Liveness too aggressive	<code>kubectl describe pod</code>	Longer thresholds; add a startup probe

Interview questions

- Name three things a StatefulSet gives that a Deployment cannot.** *Stable network identity (ordinal names plus per-pod DNS via a headless Service), stable per-replica storage (`volumeClaimTemplates`, re-attached by ordinal), and ordered startup and termination.*
- Why does a StatefulSet need a headless Service?** `clusterIP: None` makes DNS return pod IPs rather than a single virtual IP, so `postgres-0.postgres.<ns>.svc.cluster.local` resolves to that specific pod. A normal Service load-balances and cannot address one replica.

3. **You scale a PostgreSQL StatefulSet to 3. Do you have a cluster?** *No. Three independent empty databases. Kubernetes provides identity, storage and ordering; it knows nothing about replication, leader election or WAL shipping. A real cluster needs an Operator.*
 4. **Why are PVCs not deleted with the StatefulSet?** *(senior) Because deleting a workload must never silently destroy its data. It is a deliberate safety default; cleanup is explicit. Newer Kubernetes offers `persistentVolumeClaimRetentionPolicy` if you genuinely want automatic deletion.*
 5. **Your database pod is Pending after a node was drained. Why, and is Kubernetes wrong?** *(senior) Node-local storage pins the volume to one node. Kubernetes is refusing to start the database somewhere its data does not exist — which is correct. Starting it on an empty volume would look exactly like data loss. The fix is networked storage or database-level replication, not overriding the scheduler.*
-

3.5 securityContext

1. What it is

Fields on a pod or container that constrain what the process may do: which user it runs as, whether it can write to its filesystem, and which Linux capabilities it holds.

2. Why it exists

A container is a process on a shared kernel. If it runs as root and escapes, it is root on the node — and on that node's other workloads.

3. The business problem

`axispay-core` is the cardholder data environment. A container escape there is not an incident, it is a breach notification.

4. How it works

Setting	Level	Effect
<code>runAsNonRoot: true</code>	pod	The kubelet refuses to start the pod if the image's user resolves to UID 0
<code>runAsUser</code> / <code>runAsGroup</code>	pod/ container	The UID/GID the process runs as
<code>fsGroup</code>	pod	The kubelet chowns mounted volumes to this GID before start
<code>readOnlyRootFilesystem</code>	container	Root filesystem mounted read-only
<code>capabilities.drop: ["ALL"]</code>	container	Removes the ~14 capabilities containers get by default
<code>allowPrivilegeEscalation: false</code>	container	Blocks setuid binaries from gaining privileges
<code>seccompProfile: RuntimeDefault</code>	pod	Blocks ~44 dangerous syscalls

`fsGroup` **is the one that catches everyone.** A freshly provisioned volume is owned by root. A process running as UID 10001 cannot write to it, and the error points at a directory that looks perfectly normal.

5. Internal architecture

These map onto Linux primitives: user namespaces and UID/GID, cgroups, capability bounding sets, seccomp BPF filters, and a read-only bind mount for the root filesystem. Kubernetes is a configuration surface over kernel features that already existed.

`capabilities.drop: ["ALL"]` removes `NET_RAW` — enough to craft raw packets and ARP-spoof other pods on the node. A payment API needs none of the default capabilities.

6. Component interactions

<code>admission</code>	validates the spec (Pod Security Admission — Day 5)
<code>kubelet</code>	resolves the image user; REFUSES if <code>runAsNonRoot</code> and user is 0
<code>kubelet</code>	chowns volumes per <code>fsGroup</code>
<code>runtime</code>	applies capability bounding set, <code>seccomp</code> , read-only rootfs

7. Enterprise example

A bank enforces `restricted` Pod Security Admission on every namespace holding cardholder data. A workload without a compliant `securityContext` is **rejected at admission** — it never runs. Exceptions require a documented, time-boxed waiver. Defence in depth: images set `USER`, pod specs enforce it, and the namespace policy rejects anything that does neither.

8. Real-world analogy

Building access. **Defence in depth**: the employee has the right badge (the image sets `USER`), the door checks it (the pod spec enforces `runAsNonRoot`), and the floor is access-controlled regardless (Pod Security Admission). Any one alone can be circumvented; all three rarely.

9. Best practices

Practice	Reason
Set <code>USER</code> in the image and <code>runAsNonRoot</code> in the spec	Images regress; manifests get copied
<code>readOnlyRootFilesystem: true</code> + an emptyDir for <code>/tmp</code>	A compromised process cannot write a payload
<code>capabilities.drop: ["ALL"]</code> , add back only what is needed	Almost nothing needs any
<code>allowPrivilegeEscalation: false</code> always	Blocks <code>setuid</code> escalation
<code>fsGroup</code> matching the runtime GID	Or non-root cannot write to volumes
Use ports above 1024	Avoids needing <code>NET_BIND_SERVICE</code>
Enforce at namespace level	Day 5 — so forgetting is not possible

10. Common mistakes

Mistake	Symptom
Only setting <code>USER</code> in the image	A future build regresses it silently
<code>readOnlyRootFilesystem</code> with no writable temp	App crashes writing a temp file
Forgetting <code>fsGroup</code>	<code>permission denied</code> on a fresh volume
Wrong UID for the image	Postgres uses 999 , not 10001
Dropping capabilities a workload needs	<code>Operation not permitted</code> binding port 80
Assuming non-root is enough	setuid binaries can still escalate

11. Security considerations

Threat	Control	Residual risk
Container escape as root	<code>runAsNonRoot</code> + drop ALL + seccomp	Kernel vulnerabilities remain
Malware written to disk	<code>readOnlyRootFilesystem</code>	tmpfs is still writable
Privilege escalation via setuid	<code>allowPrivilegeEscalation: false</code>	—
Packet sniffing / ARP spoofing	Drop <code>NET_RAW</code>	A privileged sidecar could reintroduce it
Host filesystem access	Forbid <code>hostPath</code> (Day 5)	Some agents legitimately need it

12. Performance considerations

Essentially free. `seccompProfile: RuntimeDefault` adds a small per-syscall filter cost, immaterial for a network service. `fsGroup` chowns the volume at mount — on a very large volume with millions of files that can be slow, which is what `fsGroupChangePolicy: OnRootMismatch` addresses.

13. High availability

No direct effect. Indirectly relevant: a compromised container can degrade or destroy a node's other workloads, so hardening limits blast radius.

14. Disaster recovery

`securityContext` is configuration and recovers with the manifest. The DR-relevant point is a **compromised image**: recovery means rebuilding from a known-good base and rotating every credential the workload could reach — which is far easier if the workload could only reach a few.

15. Monitoring

Signal	Why
Pods rejected for <code>runAsNonRoot</code>	Something is trying to run as root
Pod Security Admission violations (Day 5)	Policy drift
Audit log: pods with <code>privileged: true</code>	Should be a very short list
Read-only filesystem write errors	Missing <code>emptyDir</code>

16. Troubleshooting

Symptom	Cause	Command	Fix
<code>container has runAsNonRoot</code> and image will run as root	Image has no <code>USER</code>	<code>docker inspect</code> <code>\ grep User</code>	Add <code>USER 10001</code>
Read-only file system	Writing outside a mount	<code>kubectl logs</code>	<code>emptyDir</code> for that path
permission denied on a volume	No <code>fsGroup</code>	<code>kubectl exec --</code> <code>ls -ld</code>	Set <code>fsGroup</code>
Operation not permitted on port 80	<code>NET_BIND_SERVICE</code> dropped	<code>grep CapEff /</code> <code>proc/1/status</code>	Use a port above 1024
Postgres will not start non-root	Wrong UID	<code>kubectl logs</code>	The postgres image uses 999

Interview questions

- What does `fsGroup` do?** *It makes the kubelet chown mounted volumes to that GID before the container starts, so a non-root process can write to a freshly provisioned volume. Without it you get permission denied on a directory that looks fine.*
- Why enforce non-root in the pod spec when the image already sets `USER`?** *Defence in depth. A future image build can regress, a base image can change, and a manifest can be copied without its `securityContext`. With both, a careless change is caught by one of them.*
- Why does `allowPrivilegeEscalation: false` matter if you already run as non-root?** *(senior) Because a `setuid` binary inside the container can gain privileges the parent process did not have. Running as UID 10001 does not prevent `execve` of a `setuid-root` binary; this flag sets `no_new_privs`, which does.*
- What does `capabilities.drop: ["ALL"]` actually remove?** *(senior) The ~14 capabilities containers get by default — including `NET_RAW` (raw packets, ARP spoofing), `CHOWN`, `SETUID`, `SETGID` and `MKNOD`. A typical web service needs none of them. Verify with `grep CapEff /proc/1/status`; hardened should read all zeroes.*

3.6 Database Initialization Patterns for Java Applications

1. What it is

A set of patterns for creating or changing a database schema before a Java application starts serving traffic: migrations, seed data, compatibility checks and one-time bootstrap work.

For AxisPay's Spring Boot services, that usually means Flyway or Liquibase running against PostgreSQL before `core-service` starts accepting payment writes.

2. Why it exists

A container can start in seconds. A database schema change can take minutes, can fail halfway, and is shared by every replica of the application. That mismatch is what creates outages.

3. The business problem

At 08:55 on a Monday, AxisPay rolls out `core-service` v3.6 with a new `ledger_entries.reference_type` column. The Deployment is scaled to four replicas for the morning peak. Every pod starts together. Because Flyway auto-run is enabled in Spring Boot, every JVM opens a connection and tries to apply the same migration at once.

One pod acquires the migration lock. Two wait. The fourth times out waiting for a JDBC connection because the database is already busy replaying WAL after a failover. Kubernetes sees a startup failure, restarts the pod, which tries again. Meanwhile readiness stays false, the Deployment stalls, and the traffic spike that triggered the rollout is now hitting the old version and the new version in an inconsistent state.

Nothing is technically "wrong" with Flyway. The problem is that **schema change orchestration was left to replica startup timing**, which is not an operational plan.

4. How it works

There are four common patterns:

Pattern	Where migration runs	Strength	Weakness
Spring Boot auto-run	In the main app container during JVM startup	Simple for local dev	Every replica may try at once; rollout timing and schema timing are coupled
Init container	In the pod, before the main container starts	Guarantees app container waits	Every pod still runs the init step unless you add locking
Kubernetes Job	As a separate one-shot workload	Clear separation, auditable, retryable	The rollout must wait for the Job result
Database operator / release pipeline step	Outside the app pods	Best production control	More moving parts; requires platform discipline

AxisPay's rule: local development may use Spring Boot auto-run. Shared environments and production do not. Production schema changes for `core-service` run from a **Kubernetes Job** or a carefully controlled init container that is gated by an advisory lock and executed exactly once per release.

A few details matter:

Flyway records applied versions in `flyway_schema_history`; Liquibase uses `DATABASECHANGELOG` and `DATABASECHANGELOGLOCK`; PostgreSQL advisory locks add cluster-wide mutual exclusion. If a migration fails, a failed Job is far easier to reason about than a Deployment full of crashing application pods.

5. Internal architecture

The recommended control flow is simple: a release starts a migration Job, the Job connects with a dedicated migration role, PostgreSQL grants a shared lock point, Flyway or Liquibase records the applied version, and only then does the Deployment continue. `core-service` pods run an init check for the expected schema version and start with

```
spring.flyway.enabled=false.
```

Concern	Owner
DDL execution	migration Job
Rollout ordering	release pipeline + Deployment
Mutual exclusion	lock table + advisory lock
App startup	init check + main container

6. Component interactions

```
release pipeline    applies Job/core-schema-migrate
Job pod            opens JDBC connection to PostgreSQL
PostgreSQL         grants advisory lock to one runner
Flyway/Liquibase   checks metadata table; applies pending migrations
Job pod            exits 0
Deployment rollout  creates new core-service pods
init container     verifies expected schema version exists
main container     starts JVM with Flyway auto-run disabled
readiness probe    only then adds the pod to Service endpoints
```

If the Job fails, the rollout stops before any new application pod starts writing against a half-upgraded schema.

7. Enterprise example

AxisPay treats the `core-service` ledger schema as release-controlled infrastructure, not as a startup side effect. The migration must be attributable, repeatable and visibly complete before application rollout continues. A standalone Job gives that audit trail.

Below is a full example of the pattern AxisPay prefers for `core-service` :

```

apiVersion: batch/v1
kind: Job
metadata:
  name: core-schema-migrate
  namespace: axispay-data
spec:
  backoffLimit: 3
  ttlSecondsAfterFinished: 86400
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: flyway
          image: flyway/flyway:10.17.0
          env:
            - name: FLYWAY_URL
              value: jdbc:postgresql://postgres.axispay-data.svc.cluster.local:5432/axispay
            - name: FLYWAY_USER
              valueFrom:
                secretKeyRef:
                  name: axispay-db-migration
                  key: username
            - name: FLYWAY_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: axispay-db-migration
                  key: password
          command:
            - sh
            - -c
            - |
              psql "$FLYWAY_URL" -c "SELECT pg_advisory_lock(42424242);" && \
              flyway -connectRetries=60 -baselineOnMigrate=true migrate ; \
              rc=$? ; \
              psql "$FLYWAY_URL" -c "SELECT pg_advisory_unlock(42424242);" ; \
              exit $rc
      ---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: core-service
  namespace: axispay-core
spec:
  replicas: 4
  selector:
    matchLabels:
      app: core-service
  template:
    metadata:
      labels:
        app: core-service
    spec:
      initContainers:
        - name: wait-for-schema
          image: postgres:16
          env:

```

```

- name: PGPASSWORD
  valueFrom:
    secretKeyRef:
      name: axispay-db-app
      key: password
command:
- sh
- -c
- |
  until psql \
    -h postgres.axispay-data.svc.cluster.local \
    -U axispay_app \
    -d axispay \
    -tAc "SELECT 1 FROM flyway_schema_history WHERE success = true AND version
= '2026.08.14.1' | grep -q 1; do
    echo "waiting for schema version 2026.08.14.1";
    sleep 5;
  done
containers:
- name: core-service
  image: ghcr.io/axispay/core-service:3.6.0
  env:
    - name: SPRING_FLYWAY_ENABLED
      value: "false"
    - name: SPRING_DATASOURCE_URL
      value: jdbc:postgresql://postgres.axispay-data.svc.cluster.local:5432/axispay
    - name: SPRING_DATASOURCE_USERNAME
      valueFrom:
        secretKeyRef:
          name: axispay-db-app
          key: username
    - name: SPRING_DATASOURCE_PASSWORD
      valueFrom:
        secretKeyRef:
          name: axispay-db-app
          key: password

```

The Job owns DDL; the init container only confirms that the required schema version exists.

8. Real-world analogy## 8. Real-world analogy

Renovating a bank branch while staff are arriving for work.

- Spring Boot auto-run means every employee who reaches the door first is allowed to decide whether to knock down a wall.
- An init container means each employee waits in reception until the building is ready.
- A migration Job means the renovation crew arrives first, signs the permit, finishes the work, and only then are staff let in.

9. Best practices

Practice	Reason
Disable automatic Flyway/Liquibase execution in production app pods	Replica startup is not a safe orchestration mechanism
Use a dedicated migration Job per release	Clear logs, retries, ownership and rollback point
Guard with a database-level lock	Prevents parallel runners from competing
Keep migrations small and forward-only	Easier to reason about during live traffic
Separate migration credentials from app credentials	DDL rights should be rarer than DML rights
Make app and schema changes backward compatible for at least one rollout	Old and new pods may overlap during rolling deploys
Record the expected schema version in the release notes	Operations needs a known-good checkpoint
Run the same migration in a staging restore first	Schema work is code; test it against realistic data

Backward compatibility is the subtle one. If version N of `core-service` writes a new nullable column and version N-1 ignores it, a rolling update is safe. If version N drops a column that N-1 still reads, the rollout itself becomes the outage.

10. Common mistakes

Mistake	Symptom
Letting every replica auto-run Flyway	Stalled rollouts, lock contention, noisy startup failures
Running DDL in an init container on a 10-replica Deployment with no lock	Ten pods all attempt the same migration
Using the app's normal JDBC user for DDL	Broad privileges on every pod
Combining destructive schema changes with application rollout in one step	Old pods crash while new pods are still coming up
Treating Flyway lock waits as harmless	Connection pool exhaustion while pods wait
Writing down-migrations for ledger tables and trusting them	Data semantics rarely roll back cleanly
Seeding large reference data through migrations	Startup time explodes; migration window becomes unpredictable

11. Security considerations

Threat	Control	Residual risk
App pod can alter schema	Separate migration role; app role denied DDL	Compromised migration Job still has broad rights
Secret with migration credentials leaked	Namespace-scoped Secret + short-lived credential if possible	Anyone with <code>get secrets</code> in that namespace can still read it
Unreviewed SQL in a migration	Code review and DBA approval for ledger changes	Humans can still approve bad SQL
Init container image includes shell tooling	Minimal images, signed provenance	A larger image increases attack surface

A production migration runner is highly privileged. Treat it like a controlled admin action, not a normal application side effect.

12. Performance considerations

DDL competes with live traffic for locks, CPU, I/O and WAL bandwidth. AxisPay therefore treats metadata-only changes differently from heavy rewrites and schedules anything large as its own maintenance event.

13. High availability

The HA win is isolation: a failed migration Job leaves the current ReplicaSet serving. A failed startup migration turns the rollout itself into the outage.

14. Disaster recovery

AxisPay mandates **forward-only migrations for ledger schema changes**. A `down` migration can reverse DDL, but it cannot safely unwind financial data already written under the new schema. The production rollback plan is: stop the rollout, keep the old compatible version serving if possible, then apply a forward fix or restore to a known-good point-in-time copy and reconcile transactions from event logs.

15. Monitoring

Signal	Why
Job <code>Failed</code> or repeated retries	Migration did not complete
Deployment stuck after release	Schema gate may be blocking startup
Long-running DDL	Risk to live traffic
Schema version lag	Wrong release order or wrong target DB

16. Troubleshooting

Symptom	Cause	Fix
New pods stuck in init	Expected schema version missing	Fix or rerun the migration Job
Job fails with duplicate version	Migration already applied elsewhere	Reconcile history; do not force-apply blindly
Old pods crash mid-rollout	Schema change broke backward compatibility	Use expand-and-contract over multiple releases
Migration logically wrong	Correctness bug, not pod bug	Apply a forward fix or restore and reconcile

Interview questions

- 1. Why is letting Spring Boot run Flyway on startup risky in Kubernetes?** *Because Kubernetes may start several replicas at once. Every replica then competes to change the same shared schema, consuming connections, waiting on locks, and coupling rollout success to migration timing. One lock holder and three waiting pods is still a production incident if those pods are required for capacity.*
- 2. What problem does an init container solve, and what problem does it not solve?** *It guarantees the main container will not start until the init work succeeds. It does not, by itself, guarantee that only one pod performs that work. On a multi-replica Deployment, every pod gets its own init container unless you add a shared lock or move the work to a Job.*
- 3. Why prefer a Job for production migrations?** *(senior) Because it cleanly separates schema change from app startup, gives an auditable one-shot execution record, retries independently, and fails in one place. A failed Job is operationally cheaper than a Deployment full of crashing pods.*
- 4. Why are forward-only migrations mandated for ledger schema changes?** *(senior) Because financial data written under a new schema cannot always be represented safely in the old schema. A theoretical down-migration can reverse DDL, but not the meaning of money movements already recorded. The safer rollback is usually to stop, keep serving on the old compatible app if possible, and apply a forward fix or restore to a point-in-time copy with reconciliation.*
- 5. How do you make rolling deploys safe when both old and new app versions may run together?** *(senior) Use backward-compatible expand-and-contract schema changes. Add structures first, let both versions tolerate them, switch reads and writes in later releases, and only remove old structures when no running code depends on them.*

3.7 Connection Pooling for Java Database Clients

1. What it is

Connection pooling keeps a small reusable set of JDBC connections open so a Java application does not pay the cost of creating a new PostgreSQL connection for every request.

In Spring Boot, the default pool is **HikariCP**.

2. Why it exists

Opening a database connection is expensive: TCP handshake, TLS negotiation if enabled, authentication, backend process allocation in PostgreSQL, and session setup. A payment API receiving hundreds of requests per second cannot afford that overhead per request.

The trap is assuming the answer is therefore "make the pool huge". It is not.

3. The business problem

AxisPay's midday merchant campaign sends transaction volume 3x above forecast. Horizontal Pod Autoscaler scales `core-service` from 2 replicas to 5. Each pod carries the default `maximumPoolSize=20`. What looked like a harmless scale-out has quietly reserved capacity for **100 database connections** from one service alone.

PostgreSQL is shared by `core-service`, `payment-service`, settlement jobs and a reporting task. The database `max_connections` is 120. As the fifth pod starts and the other services open routine admin and background connections, the server crosses the limit. Suddenly perfectly healthy services fail not because their SQL is bad, but because **no new backend process can be created**.

The first log line seen by merchants is not about autoscaling or HikariCP. It is this:

```
org.postgresql.util.PSQLException: FATAL: sorry, too many clients already
```

4. How it works

HikariCP hands out connections from a bounded pool.

Property	Meaning	Operational effect
<code>maximumPoolSize</code>	Upper bound on total connections in the pool	The main capacity dial
<code>connectionTimeout</code>	How long a thread waits for a connection before failing	Too low causes noise; too high hides saturation
<code>idleTimeout</code>	How long to keep excess idle connections before closing them	Controls how sticky burst capacity is
<code>leakDetectionThreshold</code>	Logs when a connection is checked out too long	Helps find forgotten closes / slow transactions
<code>minimumIdle</code>	Number of idle connections Hikari tries to keep ready	Often best left equal to a small steady-state value
<code>maxLifetime</code>	Maximum age of a connection before retirement	Should be shorter than network/device idle limits

The sizing principle is simple and often surprising:

Throughput improves until the database CPU and storage are busy enough. Beyond that point, extra connections mostly add queueing, memory use and context switching.

PostgreSQL uses a **process per connection** model, so extra connections add memory use, scheduler work, lock competition and context switching. The right mental model is a **shared connection budget**, not a per-pod entitlement.

5. Internal architecture

Think in two layers:

Layer	What it does
Application pool (HikariCP)	Reuses JDBC sessions inside each JVM
Database/proxy pool (PostgreSQL or PgBouncer)	Limits backend connection creation and multiplexes client demand

HikariCP is an application-local queue plus a set of live sockets. When every connection is busy, request threads block until one returns or `connectionTimeout` expires. That is already a form of backpressure.

The problem in Kubernetes is multiplication:

```
total potential database connections
= sum(maximumPoolSize for every replica of every service)
```

For a single service:

```
total = replicas × maximumPoolSize
```

So the innocent configuration below is not innocent at all:

Service	Replicas	maximumPoolSize	Potential connections
core-service	5	20	100
payment-service	3	10	30
merchant-service	2	5	10
batch jobs / admin	—	—	10
Total			150

If PostgreSQL `max_connections=120`, the outage is not theoretical.

A practical sizing heuristic is to start from a **global active-connection budget** of roughly `((DB CPU cores * 2) + effective spindle count)`, reserve headroom for admin and jobs, and divide the rest across services. On modern SSD-backed PostgreSQL the exact number is less important than the principle: **small fixed pools, then measure.**

6. Component interactions## 6. Component interactions

```
merchant request    -> core-service thread
Spring transaction  -> asks HikariCP for a connection
HikariCP            -> returns idle connection OR waits up to connectionTimeout
PostgreSQL          -> serves query on one backend process
HikariCP            -> connection returned to pool
HPA                 -> adds replicas during load
replica count       -> multiplies total pool size against the same database
```

Add PgBouncer and the picture changes:

```
app pod -> HikariCP -> PgBouncer -> smaller set of PostgreSQL server connections
```

Many client connections can then be multiplexed onto fewer database backends, especially in transaction-pooling mode.

7. Enterprise example

AxisPay hit this during a merchant campaign when HPA scaled `core-service` from 2 to 5 replicas while every pod still carried `maximumPoolSize=20`. That one scale-out created demand for 100 PostgreSQL connections from `core-service` alone and pushed the shared database over `max_connections`, breaking fresh connections for `payment-service` and scheduled jobs too.

Representative logs looked like this:

```
com.zaxxer.hikari.pool.HikariPool      : HikariPool-1 - Connection is not available,
request timed out after 30000ms.
o.p.util.PSQLException                 : FATAL: sorry, too many clients already
o.s.jdbc.CannotGetJdbcConnectionException: Failed to obtain JDBC Connection
```

The fix was to cut `core-service` to 8 connections per replica and place **PgBouncer** in front of PostgreSQL as a shared service, so many client connections no longer implied the same number of server backends.

8. Real-world analogy## 8. Real-world analogy

A connection pool is a set of staffed teller windows, not the size of the branch lobby.

If you open too few windows, customers queue. If you open 200 windows in a small branch, you do not serve faster; you create noise, management overhead and staff getting in each other's way.

PgBouncer is the receptionist who decides which customer truly needs a teller right now and which can wait in an orderly line.

Where it breaks: a teller can often handle a whole conversation continuously. Transaction-pooling proxies may hand different transactions to different backends, which matters for session state. That is why application behaviour still matters.

9. Best practices

Practice	Reason
Size pools from a global connection budget, not by copying defaults	Replica count multiplies everything
Start smaller than instinct suggests	PostgreSQL performance usually degrades before giant pools help
Set <code>maximumPoolSize</code> per service role	Read-heavy reporting and write-heavy ledger traffic are different
Keep <code>connectionTimeout</code> finite and alert on it	Waiting forever hides saturation
Use <code>leakDetectionThreshold</code> in lower environments and during incidents	Finds code paths that hold connections too long
Set <code>maxLifetime</code> slightly below network/load-balancer idle limits	Avoids synchronized disconnect storms
Recalculate pool sizes whenever HPA limits or replica counts change	Scaling policy is part of database capacity planning
Consider PgBouncer for shared PostgreSQL clusters	Smooths backend pressure and absorbs client fan-out

A useful starting formula for one database cluster is:

```
total app pool budget = safe PostgreSQL connection budget - admin reserve - batch reserve
per-service budget   = share based on peak concurrency and priority
per-replica pool     = floor(per-service budget / max replicas)
```

That is more realistic than treating 20 as a magic number.

10. Common mistakes

Mistake	Symptom
Copying the same <code>maximumPoolSize</code> to every service	Shared database saturates unpredictably
Scaling replicas without revisiting connection math	Outage appears during successful scale-out
Setting pools huge "for headroom"	High CPU, memory bloat, many idle backends
Very long <code>connectionTimeout</code>	Requests hang for ages instead of failing fast enough to shed load
Ignoring <code>leakDetectionThreshold</code> forever	Real leaks remain invisible until peak traffic
Putting PgBouncer in session mode by default	Lower multiplexing benefit
Forgetting batch jobs and admin tools count too	Connection budget looks fine on paper, fails in reality

11. Security considerations

Threat	Control	Residual risk
Over-privileged database user shared across services	Separate DB roles per service	Shared database still creates lateral risk
Credentials copied into many pods	External secrets + scoped Secrets	A pod compromise still exposes that service's role
PgBouncer becomes a choke point	TLS, auth, NetworkPolicy, HA deployment	Proxy compromise affects many clients
Connection leak during incident debugging	Scrub logs and traces	Stack traces may still reveal JDBC URLs

Connection pooling is not just performance plumbing. It determines how widely a database credential is distributed and what blast radius a proxy failure has.

12. Performance considerations

The real tuning trade-off is simple: too small a pool makes Java threads wait; too large a pool makes PostgreSQL spend time context switching instead of executing useful queries.

AxisPay measured better results with smaller pools once the database stayed continuously busy.

13. High availability

Large eager pools create connection storms during rollouts and failovers. Modest pools plus a highly available PgBouncer service contain that risk.

14. Disaster recovery

After failover or restore, pooled sockets may all be stale. Runbooks should say whether pods are restarted, whether PgBouncer is drained first, and how reconnect storms are controlled.

15. Monitoring

Signal	Why
Hikari pending threads / timeouts	Pool saturation inside the JVM
PostgreSQL connection utilisation	Approaching hard limit
PgBouncer client/server pools	Whether multiplexing is helping
HPA replica changes	They alter connection demand immediately

16. Troubleshooting

Symptom	Cause	Fix
Connection is not available	Pool exhausted	Reduce transaction length; resize only within DB budget
too many clients already	PostgreSQL hard limit reached	Reduce pools, add PgBouncer, or raise the limit only with proof
One pod fine, five pods fail	Replica count multiplied demand	Recompute the global connection budget
Leak detection logs	Connection held too long	Fix transaction scope or close path

Interview questions

- Why is "more connections" not always better for PostgreSQL?** *Because PostgreSQL uses a process per connection. More backends consume more memory and scheduler time, and beyond a point they mostly add lock contention and context switching instead of throughput. A smaller pool can outperform a larger one.*
- How does Kubernetes make connection-pool mistakes worse?** *By multiplying them. A per-pod pool size that looks harmless becomes dangerous when HPA scales the Deployment. Five pods with a pool of 20 is 100 potential connections from one service.*

3. **What HikariCP settings matter most operationally?** `maximumPoolSize` because it defines demand placed on the database, `connectionTimeout` because it decides how long requests wait under saturation, `idleTimeout` because it controls excess idle connections after bursts, and `leakDetectionThreshold` because it exposes code paths that hold connections too long.
4. **What problem does PgBouncer solve, and what problem does it not solve?** (senior) It reduces the number of PostgreSQL backend connections required for many client connections and smooths spikes. It does not make unbounded SQL, bad transaction design or slow queries disappear. You still need right-sized application pools and efficient database work.
5. **How would you size pools for an autoscaled service?** (senior) Start from the database's safe global connection budget, reserve capacity for admin and jobs, allocate a service budget by business priority, then divide by the service's maximum replica count rather than current replicas. Otherwise a successful scale-out becomes a connection-limit outage.
-

3.8 Secret Rotation and Certificate Management

1. What it is

The set of processes used to replace sensitive credentials and cryptographic material without breaking running systems: passwords, signing keys, encryption keys and TLS certificates.

In Kubernetes, the hard part is rarely storing the next secret. The hard part is **changing trust safely while traffic continues**.

2. Why it exists

Every secret eventually becomes untrustworthy.

If rotation requires downtime, teams delay it. Delayed rotation becomes permanent rotation debt.

3. The business problem

AxisPay's `auth-service` signs JWTs used by merchants' checkout flows. The existing signing key is nine months old. Security requires rotation this week. Operations' fear is simple: if the new key starts signing immediately and old pods stop trusting the old key immediately, every token issued seconds earlier becomes invalid and merchants see mass authentication failures.

A second problem sits in `payment-service`. PCI-DSS requires annual rotation of the card-encryption key. Those encrypted PAN-derived values live at rest in PostgreSQL. Replacing the key is not only a Kubernetes change; old rows must remain decryptable while re-encryption is in progress.

4. How it works

Rotation safety comes from **overlap windows**.

Secret type	Safe rotation pattern
JWT signing key	Dual-key: sign with new, verify with both until old tokens expire
DB password	Issue new credential, let app pick it up, revoke old after all clients moved
TLS certificate	Renew before expiry; servers present new leaf cert while clients trust the same CA chain
Data-encryption key	Encrypt new data with new key, decrypt with both, re-encrypt old data, then retire old

Kubernetes Secrets help distribute material, but they do **not** solve the live-refresh problem by themselves.

Two facts matter:

1. **Environment variables are snapshots.** If a JVM read a password from `SPRING_DATASOURCE_PASSWORD`, changing the Secret does nothing until the pod restarts.
2. **Mounted secret volumes update in place, but a Java process must re-read the file.** Most Spring Boot apps do not watch certificate or key files unless deliberately coded to.

So the rotation matrix really looks like this:

Delivery method	Secret changes in Kubernetes	Running JVM sees it?
env var	yes	No , restart required
mounted file	yes	Only if the process re-reads or watches the file
external operator + app-level refresh logic	yes	Often yes, if implemented intentionally

5. Internal architecture

A rotation has three layers: the **source of truth** (Vault, KMS, CA), the **Kubernetes projection** (Secret, cert-manager Secret, External Secrets Operator), and the **application trust logic** inside the JVM. Most failed rotations happen because teams update the middle layer and forget that the running process is still holding the old value in memory.

6. Component interactions

```

security team / platform      rotates secret in source of truth
external-secrets / cert-manager updates Kubernetes Secret
kubelet                      refreshes mounted secret volume
application                   either re-reads file OR continues using old in-memory value
rolling restart               may be required if app is env-var based or not hot-reload
aware
clients                       must tolerate overlap window where old and new both verify

```

That last line is the real design work. Rotation is a trust-transition problem, not a YAML problem.

7. Enterprise example

JWT signing key rotation for `auth-service`

AxisPay uses a `kid` header in JWTs and stores two active public verification keys plus one private signing key set.

The safe sequence is:

Step	Action	Result
1	Add new key pair to <code>auth-service</code> Secret or external source	Both old and new key material available
2	Deploy <code>auth-service</code> so verification trusts both key IDs	Existing tokens remain valid
3	Switch signing to the new <code>kid</code>	New tokens use the new private key
4	Wait for the maximum old token TTL plus clock skew	No legitimate client should still hold an old token
5	Remove the old key	Rotation complete

Without step 2, rotation is an outage. Without step 4, rotation is a race condition in disguise.

Database password rotation

AxisPay uses two patterns depending on maturity:

Pattern	How it works	Trade-off
Manual rotation + rolling restart	DBA creates new password, updates Secret, rolls Deployment, revokes old password after all pods moved	Simple, but operationally manual
Short-lived credentials via External Secrets Operator	Secret manager issues credentials with TTL; operator refreshes Kubernetes Secret; apps reload or restart automatically	Better security, more platform complexity

The manual pattern is acceptable for lower-risk internal services. For `core-service`, shorter-lived credentials are strongly preferred because static passwords create long compromise windows.

Card-encryption-key rotation for `payment-service`

For PCI-sensitive data, AxisPay does **not** treat a Kubernetes Secret as the root of trust. The application receives a reference to an HSM/KMS-managed key and uses envelope encryption.

The annual rotation sequence is:

1. Provision a new KMS/HSM key version.
2. Update `payment-service` configuration so new writes use the new key version.
3. Keep old key version enabled for decryption.
4. Run a controlled background re-encryption job over stored card artefacts.
5. Verify that all rows now point to the new key version.
6. Disable then destroy the old key version according to retention policy.

This is a data migration with security implications, not just a Secret edit.

That overlap is what prevents annual rotation from becoming annual downtime.

8. Real-world analogy## 8. Real-world analogy

Changing locks on a building while staff and customers are still inside.

- JWT dual-key rotation means the new front-door key starts being issued today, but security guards still accept yesterday's keycards until they naturally expire.
- Database password rotation means swapping the alarm code after every team has been told the new one.
- Data-encryption-key rotation means not only changing the vault combination, but re-sealing every deposit box with the new combination over time.

9. Best practices

Practice	Reason
Design applications to trust old and new keys during rotation	Eliminates hard cutovers
Prefer file-based secret mounts or dynamic retrieval for rotatable material	Env vars force restarts
Use <code>kid</code> or version identifiers for signing and encryption keys	Lets multiple keys coexist cleanly
Document token TTLs and grace periods	You cannot pick the overlap window blindly
Distinguish storage from root of trust	Kubernetes Secret is a distribution mechanism, not an HSM
Test emergency rotation separately from planned rotation	The timeline and failure modes differ
For TLS, automate renewal with cert-manager where possible	Manual certificate renewal always drifts
Alert before expiration, not at expiration	Certificates and credentials should never fail by surprise

10. Common mistakes

Mistake	Symptom
Replacing a JWT signing key with no overlap	All recently issued tokens fail instantly
Rotating a DB password but not restarting env-var-based pods	Some pods keep using the old password until they reconnect
Assuming a mounted Secret change updates a running Spring Boot bean	App continues using stale in-memory value
Storing card-encryption keys only in Kubernetes Secrets	Fails PCI expectations for key custody and protection
Re-encrypting data before the app can decrypt with both keys	Old rows become unreadable
Forgetting clock skew and token cache lifetime	Random auth failures during otherwise correct rotation

11. Security considerations

Threat	Control	Residual risk
Leaked long-lived DB password	Short-lived credentials, scoped roles	Credential misuse remains possible within TTL
Stolen JWT private key	Rapid dual-key rotation, <code>kid</code> -based verification	Tokens already issued remain valid until expiry
TLS certificate expires unnoticed	cert-manager + expiry alerts	Misconfigured renewal can still fail
Encryption key exposed from app config	HSM/KMS-backed envelope encryption	Application memory remains a sensitive boundary
Secret value updated but stale pod still trusts old compromised material	Forced restart or hot-reload validation	Rolling restart takes time

12. Performance considerations

The expensive part is usually reconnect churn or background re-encryption, not the Secret update itself. For `payment-service`, re-encrypting stored data must be throttled so it does not compete with live payment traffic.

13. High availability

Availability comes from overlap: old and new keys, passwords or certificates must coexist long enough for live sessions and in-flight traffic to survive the transition.

14. Disaster recovery

DR and rotation meet when older backups still require older keys. Key-retention policy therefore has to match backup-retention policy, or a successful restore may produce unreadable data.

15. Monitoring

Signal	Why
Certificate expiry window	Preventable outage warning
Secret age vs policy	Rotation overdue
Auth or DB login failures after change	Rotation may not be complete
Re-encryption backlog	Old key cannot yet be retired

16. Troubleshooting

Symptom	Cause	Fix
Old JWTs rejected after rotation	Old verify key missing	Reintroduce it and wait out TTL
Some pods healthy, some broken after DB password change	Mixed generations or stale env vars	Complete the rolling restart
Secret file changed but app still serves old cert	JVM loaded once at startup	Restart or implement file reload
Re-encryption stalls	Batch too aggressive or old key disabled early	Re-enable old decrypt path and throttle the job

Interview questions

- Why do Kubernetes Secrets not solve secret rotation by themselves?** *Because they only distribute data. A running JVM may have read a secret once from an env var or loaded a file into memory and never look again. Safe rotation requires application logic, restarts, or hot reload behaviour in addition to Secret updates.*
- How do you rotate a JWT signing key without downtime?** *Use a dual-key pattern: deploy the new key while still trusting the old one for verification, switch signing to the new key, wait until all old tokens have expired, then remove the old key. The overlap window must cover token TTL plus clock skew.*
- What is the safest way to rotate a database password for a Spring Boot app using env vars?** *Create the new credential, update the Secret, perform a controlled rolling restart so every pod starts using it, verify new logins succeed, then revoke the old credential. Without the restart, some pods may keep the old password indefinitely until they reconnect.*

4. Why are Kubernetes Secrets alone insufficient for PCI-grade encryption keys?

(senior) Because a Secret is a cluster object readable by anyone with the right RBAC and typically decrypted in the application process. PCI-sensitive key custody usually requires HSM/KMS-backed storage, versioning, audit trails and envelope encryption. Kubernetes may hold a reference or wrapped key, but should not be the sole trust boundary.

5. What is the danger of retiring an old encryption key before re-encryption completes?

(senior) Rows still encrypted with the old key become unreadable, which can turn a planned rotation into data loss. The system must decrypt with both keys until all data has been migrated and verified.

3.9 Backup and Restore for Payment Transaction Data

1. What it is

The set of mechanisms used to preserve and recover AxisPay's payment records: volume snapshots, logical backups, write-ahead log archiving, point-in-time recovery, restore drills and post-restore reconciliation.

For a payments ledger, backup strategy is not a side topic. It is part of the system's correctness model.

2. Why it exists

PersistentVolumes make data survive a pod restart. They do **not** protect against:

- application bugs that corrupt rows,
- an operator dropping a table,
- ransomware or hostile admin activity,
- a broken migration,
- storage-system failure,
- restoring to a specific second before a bad deployment.

A ledger without tested restore is a ledger relying on luck.

3. The business problem

At 14:12, `core-service` begins returning 500s for every write. Investigation shows a batch maintenance script connected to PostgreSQL and updated ledger balances incorrectly. The database is still running, but the truth inside it is now wrong.

This is the important distinction: the disk is healthy, the PVC is Bound, the StatefulSet is Ready — and the system is still in a severe incident because **correctness**, not mere availability, has been lost.

AxisPay must restore the ledger quickly, but also must ensure that payments accepted during the restore window are neither lost nor processed twice. That requires both a database recovery plan and an event reconciliation plan.

4. How it works

A serious PostgreSQL backup strategy usually combines multiple layers:

Technique	What it captures	Strength	Weakness
<code>VolumeSnapshot</code>	Raw storage state at a point in time	Fast restore of whole volume	Crash-consistency depends on storage/app state; coarse granularity
<code>pg_dump</code>	Logical schema/data export	Portable, selective restore	Slow for large DBs; not near-zero RPO
WAL archiving	Every committed change after a base backup	Near-zero data loss with PITR	Requires disciplined storage and replay process
Replica / operator backup tooling	Integrated DB-aware backups	Best operational ergonomics	Depends on the chosen operator/platform

For AxisPay's ledger database, the target is:

Objective	Target
RPO	Near-zero, via continuous WAL shipping
RTO	Under 15 minutes for standard restore scenarios
Backup retention	Long enough to cover audit and incident-discovery windows
Restore validation	Scheduled, documented and tested

A pure snapshot-only approach is not enough because a five-minute snapshot interval still means up to five minutes of financial data loss. For payments, that is an unacceptable RPO.

5. Internal architecture

A serious restore stack combines periodic **base backups**, continuous **WAL archiving**, an off-cluster backup repository, restore automation, and an application runbook for pausing writes and reconciling post-restore events. PVCs and StatefulSets are only the lowest layer of durability.

Failure	Best first tool
Node loss	replica or volume failover
Table dropped at 14:12	point-in-time restore
Cross-region disaster	off-cluster backup repository
Logical corruption	restore plus reconciliation

6. Component interactions## 6. Component interactions

```
core-service writes payment + ledger rows
PostgreSQL records change in WAL
WAL archiver ships segments off-cluster continuously
periodic base backup captures full starting point
restore controller/operator rebuilds database from base backup
WAL replay advances to chosen recovery timestamp
core-service reconnects to restored database
fraud-service + merchant-service event logs used to reconcile in-flight transactions
```

The last line is what makes financial restore different from restoring a wiki.

7. Enterprise example

AxisPay runs quarterly DR drills against `core-service` with a scenario deliberately more difficult than a dead node.

Drill: corrupted ledger database with live merchant traffic

Scenario: a bad script corrupts balances at 14:12. The team decides to recover to 14:11:58 and then reconcile later business events.

Recovery sequence:

Step	Action	Why
1	Freeze new writes at the edge or put payment intake into maintenance mode	Prevent new divergence during restore
2	Capture current logs and event streams from <code>core-service</code> , <code>fraud-service</code> , and <code>merchant-service</code>	Preserve evidence of in-flight work
3	Restore PostgreSQL from last base backup plus WAL to 14:11:58	Recover to last known-correct point
4	Bring <code>core-service</code> up read-only first	Validate balances before allowing writes
5	Reconcile events after 14:11:58 against merchant submissions and fraud decisions	Determine which transactions were accepted but not durably posted
6	Replay only idempotent, verified missing transactions	Avoid both loss and double-processing
7	Reopen write traffic	Resume normal business

The event reconciliation is essential because real life does not pause neatly at the recovery timestamp.

Suppose merchant A submitted payment `pay_982734` at 14:12:03.

- `merchant-service` has the inbound API event.
- `fraud-service` has an approve decision.

- `core-service` may have emitted a journal-start event but lost the commit in the restored database.

If the restore simply resumes intake and later reprocesses every post-14:11:58 event blindly, a payment could be charged twice. If it ignores them, a legitimate approved payment could be lost. The reconciliation logic therefore needs an idempotency key, authoritative event history and a rule for what "already applied" means.

A restore drill that stops at "the database pod is Running again" has tested infrastructure, not financial recovery.

8. Real-world analogy## 8. Real-world analogy

A bank branch's vault records are damaged at 14:12.

- A **snapshot** is yesterday's photocopy of the ledger book.
- **WAL archiving** is the clerk's running list of every change written as it happened.
- **Restore** is re-creating the book from yesterday's copy, then replaying every legitimate entry until 14:11:58.
- **Reconciliation** is checking CCTV, teller logs and deposit slips to confirm what happened in the minute after that.

9. Best practices

Practice	Reason
Use WAL archiving for the ledger database	Snapshot intervals alone create too much potential data loss
Store backups off-cluster and in another failure domain	A cluster disaster must not take backups with it
Treat restore time as a measured SLO	RTO is a testable number, not a wish
Keep application idempotency keys indefinitely enough for replay windows	Reconciliation without idempotency is dangerous
Document who can declare read-only / maintenance mode	Restore delay often comes from decision latency
Validate backups by restoring them regularly	Untested backups are unproven assertions
Practice logical corruption scenarios, not only node-failure scenarios	The hardest incidents are data-correctness failures
Separate backup credentials and repositories from app credentials	A compromised app should not be able to delete backups

10. Common mistakes

Mistake	Symptom
Calling a PV a backup	No way to recover from corruption or deletion
Taking snapshots with no database-consistency plan	Restores start, then fail or recover to ambiguous state
Keeping backups in the same cluster only	Cluster-wide failure wipes out primary and backup
Never testing point-in-time restore	Real incident becomes the first rehearsal
Restoring the DB but not reconciling external events	Lost or duplicated payments
Assuming fraud and merchant logs are optional	No source of truth for in-flight decisions

11. Security considerations

Threat	Control	Residual risk
Backup repository compromised	Encrypt backups, separate IAM, audit access	Key compromise still matters
Restore credentials abused	Break-glass access with review	Emergency access is still privileged
Backups contain PAN-adjacent or sensitive data	Tokenisation, encryption and minimisation	Regulatory scope still follows the data
Ransomware deletes backups	Immutable or versioned object storage	Retention policy misconfiguration

Backups are copies of the crown jewels. Secure them at least as well as production.

12. Performance considerations

Backups consume I/O, network and storage bandwidth. AxisPay therefore avoids heavy logical dumps during peak settlement windows and schedules restore drills intentionally.

13. High availability

HA keeps the service up through routine failures. DR restores it after larger failures or logical corruption. Replication helps the first case; backup history is what saves the second.

14. Disaster recovery

AxisPay's ledger targets are **near-zero RPO** through continuous WAL shipping and **under 15 minutes RTO** for standard restores. The runbook restores the database, checks ledger

invariants, reconciles `merchant-service` and `fraud-service` event logs, replays only verified missing transactions idempotently, and only then reopens writes.

15. Monitoring

Signal	Why
Latest successful base backup age	Backup schedule health
WAL archive lag	RPO risk growing
Restore drill duration	RTO realism
Ledger checks after restore	Business correctness validation

16. Troubleshooting

Symptom	Cause	Fix
Restore misses recent transactions	WAL archive gap	Repair the WAL pipeline or accept an older point with approval
Restored DB boots but totals are wrong	Corruption predates restore point or replay incomplete	Choose an earlier point and reconcile again
Backup exists but cannot be restored	Real incident became the first rehearsal	Fix tooling and schedule regular drills
Duplicate charges after recovery	Replay ignored idempotency	Reverse duplicates and harden replay rules

Interview questions

- Why is a PersistentVolume not a backup?** *Because it only stores the current state. If that state is corrupted, deleted or encrypted by malware, the PV faithfully preserves the bad state. A backup is a separate historical copy with a tested restore path.*
- Why does a payments ledger usually need WAL archiving in addition to snapshots?** *Because snapshot intervals create non-trivial RPO. If snapshots occur every five minutes, up to five minutes of committed payments can be lost. WAL archiving enables point-in-time recovery with near-zero loss.*
- What is the difference between HA and DR for PostgreSQL?** *HA handles routine failures such as pod or node loss while keeping service available. DR handles larger failures or logical corruption by restoring from historical copies. Replication helps HA; it does not help if the wrong data is replicated everywhere.*
- Why must payment restore plans include reconciliation with other services?** *(senior) Because the database recovery point rarely matches the exact business timeline of inbound requests and fraud decisions. Merchant-service and fraud-service logs help determine which transactions were accepted, approved or already posted so missing ones can be replayed idempotently and duplicates avoided.*

- 5. What does “untested backups are not backups” mean operationally? (senior)** *It means a successful backup job proves only that data was written somewhere, not that the organisation can restore it within RTO and with correct business semantics. The restore drill is what validates the backup strategy.*
-

3.10 AxisPay Secrets Inventory: What Lives Where

AxisPay now has enough moving parts that saying "put it in a Secret" is not a serious design answer. Different secret types have different owners, lifetimes, blast radii and compliance requirements.

The inventory below is the practical map operations uses.

Secret category	Primary service	Kubernetes form	Source of truth	Rotation pattern	Protection notes
PostgreSQL app passwords	core-service , payment-service , merchant-service	Secret projected by External Secrets Operator	External secret manager / database credential system	Prefer short-lived; otherwise rolling restart after update	Namespace-scoped Secret, least-privilege DB role, etcd encryption at rest
PostgreSQL migration password	migration Job for core-service	Separate Secret in axispay-data	External secret manager	Per release window or short TTL	Broader DDL rights; tighter RBAC than app secrets
JWT signing keys	auth-service	Secret or mounted key file	Central key-management process	Dual-key rotation with overlap	kid required; old verify key retained until tokens expire
Card-encryption keys / wrapped DEKs	payment-service	Reference or wrapped material in Secret	HSM/KMS	Annual minimum, emergency on compromise	Kubernetes Secrets alone are insufficient for PCI-DSS Level 1; use envelope encryption
Merchant API keys	merchant-service	Per-namespace Secret objects	Merchant onboarding / secret manager	On merchant request, scheduled rotation, or compromise	Strict RBAC; avoid cross-namespace sharing
TLS certificates	ingress / service endpoints	cert-manager-managed Secret	Internal or public CA	Auto-renew before expiry	Watch renewal and reload behaviour
Third-party gateway credentials	payment-service	Secret from external manager	Vendor portal / enterprise secrets platform	Vendor schedule + internal policy	Separate by provider to limit blast radius
Redis auth token	fraud-service	Secret	External secret manager or manual bootstrap	Scheduled + on staff turnover	Mounted as file preferred over env

A few patterns stand out immediately.

First, **base64 is not encryption**. Every value under `.data` in a Kubernetes Secret is only encoded for transport. Anyone with permission to read the object can decode it instantly. That means a Secret is useful, but only as one layer in a stack:

Layer	Why it matters
RBAC	Decides who may read the Secret at all
etcd encryption at rest	Protects backups and disk contents of the control plane
Namespace isolation	Prevents unrelated teams from seeing each other's credentials
External secrets manager	Gives rotation, audit trail and shorter-lived credentials
Application design	Avoids leaking secrets into logs, stack traces and env dumps

Second, not every sensitive value should live in Kubernetes as the **authoritative** copy.

For `core-service`, the database password may be projected into Kubernetes because the app needs it. But the source of truth should ideally be a system that can rotate it and audit access. That is why AxisPay prefers External Secrets Operator for database credentials. Kubernetes becomes the delivery mechanism, not the safe.

For `auth-service`, JWT signing keys sit in a more delicate category. A leaked password lets an attacker log in as the app. A leaked private signing key lets them mint identities for everyone. That is why the rotation schedule is explicit, the `kid` header is mandatory, and the old key is retained only long enough for verification overlap.

`payment-service` is stricter again. Card-encryption keys cannot be treated like ordinary configuration. PCI-DSS Level 1 expectations are not satisfied by saying "the Secret is in etcd and etcd is encrypted". The correct pattern is **envelope encryption**:

1. data is encrypted with a data-encryption key,
2. that key is wrapped by an HSM/KMS-managed master key,
3. the app gets only the wrapped material or a key reference,
4. the actual root key never lives as plain text in Kubernetes.

That is a very different trust model from a simple JDBC password.

Merchant API keys create a tenancy problem, so AxisPay scopes them tightly and uses RBAC to keep `get secrets` permissions off most operational roles. TLS certificates add a separate reload problem: a renewed Secret does nothing if the terminating process does not reload it.

The inventory also answers a governance question: **what should never be copied into ConfigMaps, container images or Git?**

Never store here	Why
ConfigMap	No sensitive-data boundary at all
Image layer	Secret is baked forever into the artifact and registry history
Plain Git	Secret history is effectively permanent
Shared namespace-wide mega-Secret	Any pod needing one value gets all of them

The practical takeaway is simple: Kubernetes Secrets are necessary, but insufficient. You still need RBAC, etcd encryption at rest, external secret management where possible, application-aware rotation, and KMS/HSM integration for the most sensitive cryptographic material.

When someone asks "where does this secret live?" the correct answer at AxisPay is now a full path: source of truth, Kubernetes projection, runtime consumer, rotation schedule and recovery plan.

Day 3 cheat sheet

Config and secrets

```
kubectl get configmap,secret -n axispay-core
kubectl describe configmap axispay-platform-config -n axispay-core

# decode a Secret - base64 is ENCODING, not encryption
kubectl get secret axispay-db-credentials -n axispay-data \
  -o jsonpath='{.data.POSTGRES_PASSWORD}' | base64 -d; echo

# env vars are a SNAPSHOT - a change needs a restart
kubectl rollout restart deployment/<name> -n <ns>

# mounted volumes update in place (~60s) - but subPath NEVER does
kubectl exec -n <ns> <pod> -- cat /etc/axispay/rules/fraud-rules.yaml
kubectl exec -n <ns> <pod> -- mount | grep secrets    # tmpfs
```

Storage

```
kubectl get sc,pv,pvc -A
kubectl describe pvc <name> -n <ns>          # the EVENT is everything

# is Pending normal or broken?
#   "waiting for first consumer"           -> normal, create a pod
#   "storageclass ... not found"           -> broken
#   "no persistent volumes available"      -> broken

kubectl get pv -o custom-columns=\
NAME:.metadata.name,CAP:.spec.capacity.storage,\
RECLAIM:.spec.persistentVolumeReclaimPolicy,STATUS:.status.phase,CLAIM:.spec.claimRef.name
```

Access mode	One...
<code>ReadWriteOnce</code>	NODE (not pod)
<code>ReadWriteOncePod</code>	pod, genuinely
<code>ReadWriteMany</code>	needs NFS/CephFS — most block storage cannot

StatefulSets

```
kubectl get statefulset,pvc -n axispay-data
kubectl get svc postgres -n axispay-data          # CLUSTER-IP must be None

# stable per-pod DNS
nslookup postgres-0.postgres.axispay-data.svc.cluster.local

# PVC naming: <template>-<statefulset>-<ordinal>
#   data-postgres-0, data-postgres-1 ...
# NOT deleted when the StatefulSet is deleted — deliberate.
```

Database

```
kubectl exec -it -n axispay-data postgres-0 -- psql -U axispay_app -d axispay
```

```
SELECT COUNT(*) FROM payments;                -- 5000
SELECT * FROM v_ledger_balance;                -- imbalance must be 0
SELECT COUNT(*) FROM payments WHERE amount_minor <> fee_minor + net_minor; -- 0
SELECT currency, COUNT(*), SUM(amount_minor)/100.0 FROM payments GROUP BY 1;
```

securityContext

```
kubectl exec -n <ns> <pod> -- id                # uid=10001
kubectl exec -n <ns> <pod> -- grep CapEff /proc/1/status # 0000000000000000
kubectl exec -n <ns> <pod> -- sh -c 'touch /x'    # Read-only file system
```

Control	Prevents
<code>runAsNonRoot</code>	Escape becoming root on the node
<code>readOnlyRootFilesystem</code>	Writing a payload to disk
<code>capabilities.drop: ALL</code>	Raw packets, ARP spoofing, setuid, mknod
<code>allowPrivilegeEscalation: false</code>	setuid escalation
<code>fsGroup</code>	<i>(not security — makes volumes writable)</i>

Day 3 review questions

1. Change a ConfigMap. Do running pods see it? Both cases.
2. Why does a `subPath` mount never update?
3. Is a Kubernetes Secret encrypted? What actually protects it?
4. Name three ways a Secret can leak, and a control for each.
5. Why prefer a Secret volume over an environment variable?
6. What does `ReadWriteOnce` actually mean?
7. Which access mode genuinely guarantees a single writer?
8. Explain `WaitForFirstConsumer` by describing the failure it prevents.
9. `Retain` vs `Delete` — which for a payments ledger, and why?
10. What does `fsGroup` do, and what breaks without it?
11. Name three guarantees a StatefulSet gives that a Deployment cannot.
12. Why does a StatefulSet need a headless Service?
13. You scale PostgreSQL to 3 replicas. Do you have a cluster?
14. Why are PVCs not deleted with the StatefulSet?
15. Why is `PGDATA` a subdirectory of the mount point?
16. Why enforce non-root in the pod spec when the image sets `USER`?
17. What does `capabilities.drop: ["ALL"]` remove, and how do you verify it?
18. Your PVC is Pending. How do you tell "normal" from "broken"?
19. Is a PersistentVolume a backup?
20. Why is the `payments_balance` CHECK in the schema rather than in the application?

Answers: `documents/assessments/answer-keys/day3-answer-key.md`

Day 3 summary

You built: two ConfigMaps replacing config in eleven manifests · five Secrets with the JWT key out of plaintext · a StorageClass with `Retain` and `WaitForFirstConsumer` · a static PV/PVC pair · three StatefulSets with persistent volumes · **5,000 payments and a ledger that balances to zero across 4,955 journals** · init containers for ordered startup · every application pod at UID 10001 with zero capabilities.

You proved: env vars are snapshots and volumes update in place · a Secret decodes in one command · a database as a Deployment loses its data and its identity · node-local storage pins a pod to a node, and Kubernetes correctly refuses to start it elsewhere · `fsGroup` is what makes a non-root workload able to write.

And you fixed yesterday's bug. Fraud velocity counters are in Redis, so the threshold means what it says at any replica count.

What is still missing:

Gap	Consequence	Fixed
No route in from outside	No merchant can integrate	L4.3
No TLS	Card data over plaintext HTTP	L4.3
Flat pod network	<code>axispay-edge</code> can reach PostgreSQL directly — a PCI finding	L4.4
No placement control	Three payment replicas can land on one node	L4.5
No disruption budget	A node drain takes every replica at once	L4.6
DNS still unopened	You have relied on it for three days	L4.2

Tonight (optional, 10 minutes): exec into an `edge-gateway` pod and connect to PostgreSQL directly. It works. Understanding why that is a finding — and not a feature — is the whole of Day 4.